

Pontifícia Universidade Católica de Campinas

ESTUDOS DE BANCO DE DADOS I

Prof Dra Renata Arantes

E-mail: renata.arantes@puc-campinas.edu.br

Restrições de Integridade

Restrições de integridade em bancos de dados relacionais são regras que garantem a consistência e a validade dos dados.

Elas são fundamentais para assegurar que o banco de dados reflita corretamente o modelo do mundo real.

Objetivo:

Entender as principais restrições de integridade em bancos de dados relacionais e sua importância para garantir a validade dos dados.

1. Integridade de Entidade

Os valores inseridos na coluna da chave primaria (PK) devem ser sempre únicos, não admitindo-se repetições nesses valores.

Desta forma, os registros serão sempre distintos.

Os valores de chave primaria também não podem ser nulos.

Integridade de Entidade

Definição: Nenhuma entidade pode existir sem uma identificação exclusiva.

- Como implementar: Chave primária (Primary Key).
- Exemplo prático: Em uma tabela de Clientes, cada cliente deve ter um ID único.

```
CREATE TABLE Clientes (  
  ClienteID INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  Email VARCHAR(100)  
);
```

Outro Exemplo: Imagine uma lista de alunos onde cada aluno tem um número único de matrícula. Isso impede duplicações e garante que você possa identificar univocamente cada aluno.

2. Integridade Referencial:

O que é Integridade Referencial

Garante que um valor em uma tabela filha exista antes na tabela pai.
Implementada com FOREIGN KEY.

Pai: tabela referenciada (onde está a chave primária/única).

Filha: tabela que guarda a referência (FK).

FK pode ser NULL (a menos que você defina NOT NULL).

Ação ao excluir/alterar no Pai: RESTRICT/NO ACTION, CASCADE, SET NULL, SET DEFAULT.

Integridade Referencial

- **Definição:** Garante que as relações entre tabelas sejam válidas.
- **Como implementar:** Chave estrangeira (Foreign Key).
- **Exemplo prático:** Em um relacionamento entre Pedidos e Clientes, cada pedido deve estar associado a um cliente válido

```
CREATE TABLE Pedidos (  
    PedidoID INT PRIMARY KEY,  
    ClienteID INT,  
    DataPedido DATE,  
    FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID)  
);
```

```
CREATE TABLE Clientes (  
    ClienteID INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    Email VARCHAR(100)  
);
```

Outro exemplo : Um aluno de um curso só pode se matricular em disciplinas oferecidas pela universidade. Se tentarmos registrar uma disciplina para um aluno que não existe no sistema, será um erro.

1) Básico: FK simples (RESTRICT/NO ACTION)

-- Tabela PAI

```
CREATE TABLE Cliente (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

-- Tabela FILHA

```
CREATE TABLE Pedido (  
  id INT PRIMARY KEY,  
  cliente_id INT NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  CONSTRAINT fk_pedido_cliente  
    FOREIGN KEY (cliente_id) REFERENCES Cliente(id)  
);
```

☞ Não permite inserir Pedido com cliente_id inexistente.

☞ Não permite apagar Cliente se houver Pedido associado.

1) Básico: FK simples (RESTRICT/NO ACTION)

Quando você não coloca nada, o banco assume por padrão ON DELETE RESTRICT (ou equivalente NO ACTION).

◆ O que significam:

RESTRICT

Impede excluir/alterar o registro pai se houver filhos associados.

O banco faz a verificação na hora do comando.

Exemplo: não deixa apagar um Cliente que tenha Pedidos.

NO ACTION

Parece igual, mas tecnicamente o banco só verifica no final da transação.

No dia a dia, a diferença quase não aparece (a maioria dos SGBDs trata RESTRICT e NO ACTION como se fossem a mesma coisa).

2) ON DELETE CASCADE (apaga filhos automaticamente)

```
CREATE TABLE Categoria (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Produto (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  categoria_id INT NOT NULL,  
  CONSTRAINT fk_produto_categoria  
    FOREIGN KEY (categoria_id) REFERENCES Categoria(id)  
    ON DELETE CASCADE  
);
```

☞ Ao apagar uma Categoria, todos os Produto dessa categoria são apagados.

⚡ ON DELETE CASCADE

Acontece quando a tabela filha tem a FK com a cláusula ON DELETE CASCADE.

Se você apaga um registro da tabela pai, o banco automaticamente apaga todos os registros filhos que dependem dele.

🚀 Exemplo

```
CREATE TABLE Categoria (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Produto (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  categoria_id INT NOT NULL,  
  CONSTRAINT fk_produto_categoria  
    FOREIGN KEY (categoria_id)  
    REFERENCES Categoria(id)  
    ON DELETE CASCADE  
);
```

◆ O que acontece?

1. Crio uma categoria:

```
INSERT INTO Categoria VALUES (1, 'Bebidas');
```

2. Crio produtos dessa categoria:

```
INSERT INTO Produto VALUES (10, 'Suco', 1);  
INSERT INTO Produto VALUES (11, 'Refrigerante', 1);
```

3. Agora apago a categoria:

```
DELETE FROM Categoria WHERE id = 1;
```

→ Resultado: o banco apaga automaticamente os produtos (10 e 11), porque eles estavam ligados à categoria 1.

3) ON DELETE SET NULL (mantém filho, “desliga” a referência)

```
CREATE TABLE Departamento (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Funcionario (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    departamento_id INT NULL,  
    CONSTRAINT fk_func_depart  
        FOREIGN KEY (departamento_id) REFERENCES Departamento(id)  
        ON DELETE SET NULL  
);
```

☞ Ao apagar um Departamento, o campo departamento_id dos funcionários vira NULL.

3) ON DELETE SET NULL (mantém filho, “desliga” a referência)

◆ Quando isso pode ser vantagem

O SET NULL faz sentido quando:

1. O filho pode existir sem o pai

Exemplo: funcionário continua na empresa, mas o departamento foi extinto e ele ainda não foi realocado.

Nesse período, o campo fica NULL para mostrar que está “sem alocação”.

2. Não quero perder os dados do filho

Se fosse CASCADE, eu apagaria também o funcionário → o que poderia ser um erro grave (perder histórico).

Com SET NULL, mantenho todos os dados do funcionário e depois trato essa situação.

3. Regras temporárias ou transicionais

Exemplo: um produto perde a categoria (categoria apagada), mas o produto continua sendo vendido → a categoria pode ser preenchida depois.

4) ON UPDATE CASCADE (propaga mudança de chave no Pai)

> Útil quando a PK do Pai pode mudar (não é comum, mas didático).

```
CREATE TABLE Plano (  
    codigo VARCHAR(10) PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Assinatura (  
    id INT PRIMARY KEY,  
    plano_cod VARCHAR(10) NOT NULL,  
    CONSTRAINT fk_assinatura_plano  
        FOREIGN KEY (plano_cod) REFERENCES Plano(codigo)  
        ON UPDATE CASCADE  
);
```

☞ Se Plano.codigo mudar, Assinatura.plano_cod é atualizado automaticamente.

4) ON UPDATE CASCADE (propaga mudança de chave no Pai)

◆ Passo a passo

1. Inserindo um plano:

```
INSERT INTO Plano VALUES ('10', 'Plano Básico');
```

2. Criando assinaturas ligadas ao plano 10:

```
INSERT INTO Assinatura VALUES (1, 'Ana', '10');  
INSERT INTO Assinatura VALUES (2, 'Bruno', '10');
```

3. Alterando o código do plano:

```
UPDATE Plano SET codigo = '20' WHERE codigo = '10';
```

◆ Resultado

A tabela Plano agora tem codigo = 20.

A tabela Assinatura automaticamente muda:

Ana → plano_cod = 20

Bruno → plano_cod = 20

✓ Resumindo

ON UPDATE CASCADE → se a chave do pai mudar, o banco atualiza automaticamente os filhos.

Na prática, é pouco usado, porque normalmente não se altera chave primária.

5) Tabela de Associação (N:N) com PK composta

```
CREATE TABLE Aluno (  
  ra INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Disciplina (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

-- Associação N:N

```
CREATE TABLE AlunoDisciplina (  
  ra_aluno INT NOT NULL,  
  id_disciplina INT NOT NULL,  
  semestre VARCHAR(10) NOT NULL,  
  CONSTRAINT pk_alunodisciplina PRIMARY KEY (ra_aluno, id_disciplina),  
  CONSTRAINT fk_ad_aluno FOREIGN KEY (ra_aluno) REFERENCES Aluno(ra),  
  CONSTRAINT fk_ad_disc FOREIGN KEY (id_disciplina) REFERENCES Disciplina(id)  
);
```

☞ A PK composta impede o mesmo aluno de se matricular duas vezes na mesma disciplina.

💡 O que essa modelagem garante?

Cada aluno pode se matricular em várias disciplinas.

Cada disciplina pode ter vários alunos.

Mas a PK composta (ra_aluno, id_disciplina) impede duplicação

→ não é possível o mesmo aluno se matricular duas vezes na mesma disciplina.

6) FK Composta (referencia duas colunas do Pai)

```
CREATE TABLE Turma (  
    ano      INT,  
    codigo   INT,  
    professor VARCHAR(100) NOT NULL,  
    CONSTRAINT pk_turma PRIMARY KEY (ano, codigo)  
);
```

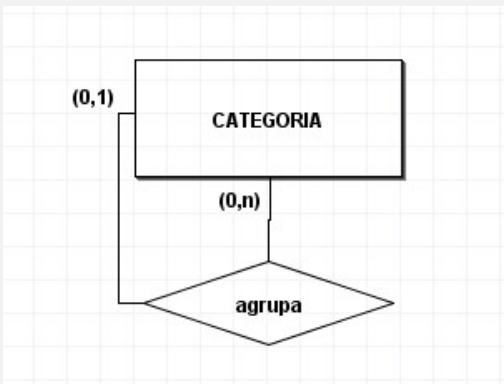
```
CREATE TABLE Matricula (  
    id       INT PRIMARY KEY,  
    aluno_ra INT NOT NULL,  
    ano      INT NOT NULL,  
    codigo   INT NOT NULL,  
    CONSTRAINT fk_matricula_turma FOREIGN KEY (ano, codigo) REFERENCES Turma(ano, codigo)  
);
```

☞ A FK usa (ano, codigo) juntos para referenciar a Turma correta.

7) Auto-referência (hierarquia na mesma tabela)

```
CREATE TABLE Categoria (  
  id          INT PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  categoria_pai INT NULL,  
  CONSTRAINT fk_categoria_pai FOREIGN KEY (categoria_pai) REFERENCES Categoria(id)  
);
```

☞ Permite árvore de categorias (categoria pode ter “mãe”).



```
INSERT INTO Categoria (id, nome, categoria_pai) VALUES  
(1, 'Eletrônicos', NULL), -- raiz  
(2, 'Celulares', 1), -- filha de 1  
(3, 'Smartphones', 2), -- filha de 2  
(4, 'TVs', 1), -- filha de 1  
(5, 'Roupas', NULL), -- outra raiz  
(6, 'Camisetas', 5); -- filha de 5
```

3. Integridade de Domínio

Garante que os valores inseridos em uma coluna sejam válidos dentro de um conjunto de regras.

É aplicada diretamente no campo (coluna).

Pode ser feita com:

Tipos de dados (INT, VARCHAR, DATE etc.)

NOT NULL (obrigar preenchimento)

DEFAULT (valor padrão)

CHECK (regras mais específicas)

UNIQUE (não permitir repetição)

3. Integridade de Domínio

✦ Exemplos simples

1. Tipo de dado + NOT NULL

```
CREATE TABLE Produto (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10,2) NOT NULL  
);
```

☞ Aqui o domínio é: preco tem que ser número decimal e não pode ser nulo.

Integridade de Domínio

- **Definição:** Os valores de um atributo devem estar dentro de um conjunto específico de valores (domínio).
- **Como implementar:** Tipos de dados e restrições adicionais (CHECK).
- **Exemplo prático:** O campo Idade de uma tabela Pessoas deve aceitar apenas números positivos.

```
CREATE TABLE Pessoas (  
    PessoaID INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    Idade INT CHECK (Idade > 0)  
);
```



2. Valores restritos (CHECK)

```
CREATE TABLE Aluno (  
    ra INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    sexo CHAR(1) CHECK (sexo IN ('M','F'))  
);
```

☞ Só aceita valores 'M' ou 'F' para sexo.

Integridade de Domínio

3. Intervalo permitido (CHECK)

```
CREATE TABLE Promocao (  
  id      INT PRIMARY KEY,  
  produto VARCHAR(100) NOT NULL,  
  desconto DECIMAL(5,2) CHECK (desconto BETWEEN 0 AND 100)  
);
```

☞ O desconto só pode estar entre 0 e 100 (%).

Integridade de Domínio

4. DEFAULT (valor padrão)

Define o valor automático de uma coluna quando o INSERT não informar essa coluna.

```
CREATE TABLE Pedido (  
  id      INT PRIMARY KEY,  
  cliente VARCHAR(100) NOT NULL,  
  total   DECIMAL(10,2) NOT NULL CHECK (total >= 0),  
  status  VARCHAR(15) NOT NULL DEFAULT 'ABERTO',    -- valor padrão  
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP -- data/hora padrão  
);
```

Integridade de Domínio

Como funciona:

```
-- Usa os DEFAULTs:  status = 'ABERTO',  criado_em = agora
INSERT INTO Pedido (id, cliente, total)
VALUES (1, 'Ana', 200.00);
```

```
-- Sobrescreve o DEFAULT de 'status'
INSERT INTO Pedido (id, cliente, total, status)
VALUES (2, 'Bruno', 180.00, 'FATURADO');
```

Pontos-chave:

DEFAULT só entra quando a coluna não é informada no INSERT.
Você pode sobrescrever o padrão informando a coluna.
Combine com NOT NULL para garantir que nunca fique vazio.

id	cliente	total	criado_em
1	Ana	200.00	2025-08-31 10:47:22

4. Integridade de Unicidade

- **Definição:** Garante que não existam valores duplicados em um ou mais atributos.
- **Como implementar:** Constrains UNIQUE.
- **Exemplo prático:** Em uma tabela de Funcionários, o número de CPF deve ser único para cada funcionário.

```
CREATE TABLE Funcionarios (  
    FuncionarioID INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    CPF VARCHAR(11) UNIQUE  
);
```

Explicação didática: Imagine que não podemos ter dois alunos com o mesmo número de matrícula. Da mesma forma, essa restrição impede duplicidade em atributos que precisam ser únicos.

4. Diferença

```
CREATE TABLE Alunos (  
  RA INT PRIMARY KEY, -- RA identifica unicamente o aluno  
  Nome VARCHAR(100),  
  Email VARCHAR(100)  
);
```

```
CREATE TABLE Alunos (  
  RA INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  Email VARCHAR(100) UNIQUE -- cada email deve ser único,  
                             mas não é a PK  
);
```

🚀 Resumo prático

PRIMARY KEY → identidade principal da linha, não pode ser nula, só pode existir uma por tabela.

UNIQUE → garante apenas que não haja duplicados naquela coluna/conjunto de colunas; aceita nulos; pode ter várias.

👉 **Então sim:** tanto PK quanto UNIQUE impedem repetição.

A diferença é que a PK é a "identidade oficial" da linha, e a UNIQUE é apenas uma restrição adicional.

5. Integridade de Vazio

Este tipo de integridade informa se a coluna é obrigatória ou opcional – ou seja, se é possível não inserir um valor na coluna.

Uma coluna de chave primaria , por exemplo, sempre deve ter dados inseridos, e nunca pode estar vazia, para nenhum registro.

O Que é NULL?

- Definição:** NULL representa a ausência de um valor ou informação. Ele não é zero, uma string vazia ou qualquer outro valor numérico ou textual; é simplesmente "nenhum valor".

- Uso:** Um campo com NULL significa que o valor é desconhecido ou não aplicável.

Integridade de Vazio – Valores Nulos (Null) – significa ausência total de dados

Um valor Null, significa que não existem dados.

É diferente de zero, espaço, string vazia ou tabulação.

Os nulos podem ser problemáticos , pois indicam:

- O valor não foi inserido
- O valor é desconhecido

```
CREATE TABLE Cliente (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100)  
);
```

```
-- email = NULL (sem valor)
```

```
INSERT INTO Cliente (id, nome, email) VALUES (1, 'Ana', NULL);
```

```
-- email = '' (string vazia, valor diferente de NULL)
```

```
INSERT INTO Cliente (id, nome, email) VALUES (2, 'Bruno', '');
```

```
-- email = 'testes@abc.com'
```

```
INSERT INTO Cliente (id, nome, email) VALUES (3, 'Carlos', 'testes@abc.com');
```

```
id | nome  | email
```

```
-----
```

```
1 | Ana   | NULL
```

```
2 | Bruno |
```

```
3 | Carlos | testes@abc.com
```

Integridade de Vazio – Exemplo de valores Null

Suponha uma tabela de cadastro de alunos.

Todo aluno deverá ter um nome cadastrado, de modo que esse campo é obrigatório (atributo não nulo).

Nem todo aluno possui telefone, portanto esse campo não é obrigatório (atributo nulo)

Exemplos de Valores NULL

ID_Aluno	Nome_Aluno	Sobrenome_Aluno	Telefone	Data_Nascimento
111	Márcio	Zanori	(11) 26533211	01/03/1983
112	Renato	Marconi	NULL	14/11/1985

Imagine uma busca: O que significa? Que todos que estão com o campo NULL , não foram demitidos.

```
SELECT * FROM Funcionarios WHERE DataDeDemissao IS NULL;
```

6. Integridade definida pelo Usuário

Diz respeito a regras de negócio específicas que são definidas pelo usuário de banco de dados.

Por exemplo: pode-se definir que uma coluna somente aceitará um conjunto restrito de valores.

Pra aquela modelagem em específico existe uma regra de negócio específica.

✓ Exemplo bem simples de integridade definida pelo usuário

O salário de um funcionário não pode ser negativo.

A idade tem que ser maior ou igual a 18.

```
CREATE TABLE Funcionario (  
  id      INT PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  idade   INT NOT NULL,  
  salario DECIMAL(10,2) NOT NULL,
```

-- Integridade definida pelo usuário (regra de negócio)

```
CONSTRAINT chk_idade_maior_18  
  CHECK (idade >= 18),
```

```
CONSTRAINT chk_salario_nao_negativo  
  CHECK (salario >= 0)
```

```
);
```

✓ Exemplo bem simples de integridade definida pelo usuário

Regra de negócio: o campo “tipo de conta” só pode ser ‘Corrente’ ou ‘Poupança’.

```
CREATE TABLE ContaBancaria (  
  id_conta  INT PRIMARY KEY,  
  titular   VARCHAR(100) NOT NULL,  
  tipo_conta VARCHAR(20) NOT NULL,  
  saldo     DECIMAL(12,2) NOT NULL DEFAULT 0,  
  
  CONSTRAINT chk_tipo_conta  
    CHECK (tipo_conta IN ('Corrente', 'Poupança')),  
  
  CONSTRAINT chk_saldo_nao_negativo  
    CHECK (saldo >= 0)  
);
```

✓ Exemplo bem simples de integridade definida pelo usuário

✦ Exemplos Clássicos de CHECK

1. Idade mínima

CHECK (idade >= 18)

☞ Exige que o funcionário/aluno seja maior de idade.

2. Salário não negativo

CHECK (salario >= 0)

☞ Impede cadastro de salário negativo.

3. Data coerente

CHECK (data_fim >= data_inicio)

☞ Garante que a data de término não seja anterior à data de início.

4. Lista de valores permitidos

CHECK (sexo IN ('M','F'))

☞ Restringe a coluna a opções pré-definidas.

5. Percentual válido

CHECK (desconto BETWEEN 0 AND 100)

☞ Impede desconto menor que 0% ou maior que 100%.

✓ Exemplo bem simples de integridade definida pelo usuário

✦ Código

1. Idade mínima

CHECK (idade >= 18)

☞ Exige que o funcionário/aluno seja maior de idade.

2. Salário não negativo

CHECK (salario >= 0)

☞ Impede cadastro de salário negativo.

.

1- Idade mínima

```
CREATE TABLE Pessoa (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    idade INT NOT NULL,  
    CONSTRAINT chk_idade_maior_18 CHECK (idade >= 18)  
);
```

2 - Salário não negativo

```
CREATE TABLE Funcionario (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    salario DECIMAL(10,2) NOT NULL,  
    CONSTRAINT chk_salario_positivo CHECK (salario >= 0)  
);
```

✓ Exemplo bem simples de integridade definida pelo usuário

✈ Códigos

3. Data coerente

CHECK (data_fim >= data_inicio)

☞ Garante que a data de término não seja anterior à data de início);

4. Lista de valores permitidos

CHECK (sexo IN ('M','F'))

☞ Restringe a coluna a opções pré-definidas.

5. Percentual válido

CHECK (desconto BETWEEN 0 AND 100)

☞ Impede desconto menor que 0% ou maior que 100%.

3. Datas coerentes

```
CREATE TABLE Evento (  
  id      INT PRIMARY KEY,  
  descricao VARCHAR(100) NOT NULL,  
  data_inicio DATE NOT NULL,  
  data_fim   DATE NOT NULL,  
  CONSTRAINT chk_datas_coerentes CHECK (data_fim >= data_inicio)
```

4. Lista de valores permitidos

```
CREATE TABLE Aluno (  
  ra      INT PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  sexo    CHAR(1) NOT NULL,  
  CONSTRAINT chk_sexo_valido CHECK (sexo IN ('M','F'))  
);
```

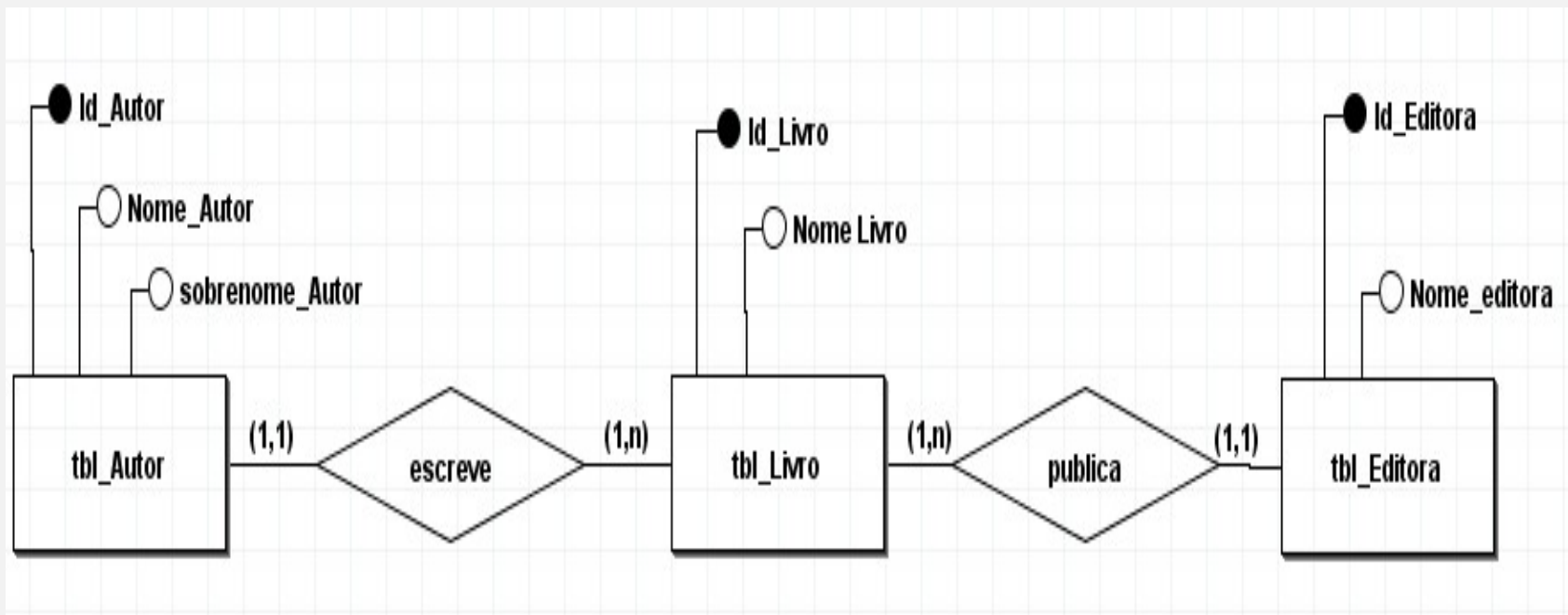
5. Percentual válido

```
CREATE TABLE Promocao (  
  id      INT PRIMARY KEY,  
  produto VARCHAR(100) NOT NULL,  
  desconto DECIMAL(5,2) NOT NULL,  
  CONSTRAINT chk_desconto_valido CHECK (desconto BETWEEN 0 AND 100)  
);
```

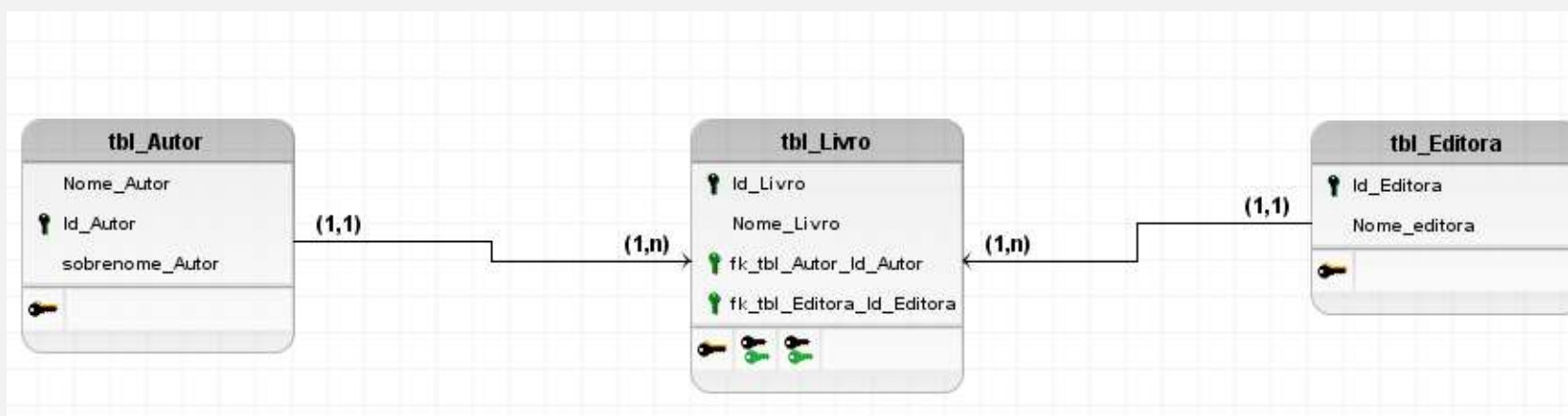
Dicionário de Dados

Um dicionário de dados é um documento usado para armazenar informações sobre o conteúdo , formato e a estrutura de uma banco de dados, assim como os relacionamentos entre os seus elementos

É importante manter um dicionário de dados para limitar erros ao criar a estrutura física do banco de dados no computador.



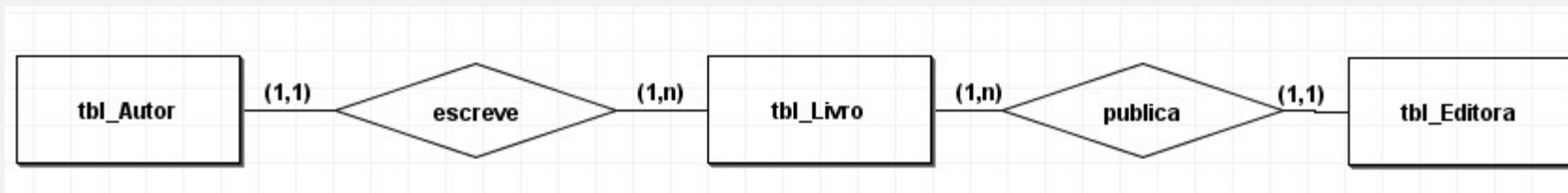
Conceitual



Lógico

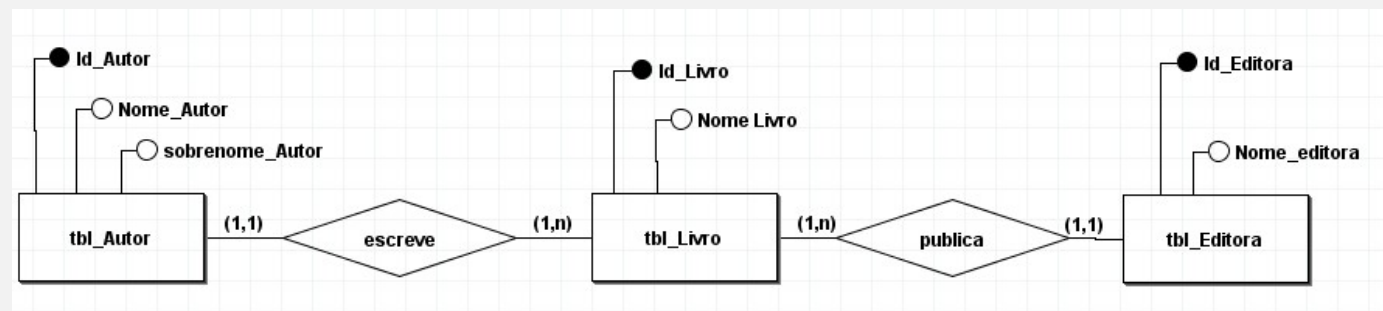
Exemplo de Dicionário de Dados

Tabela	Relacionamento	Nome do Relacionamento	Descrição
tbl_Livro	tbl_Autor	Escreve	Tabela para cadastro dos livros da coleção
	tbl_Editora	Publica	
tbl_Autor	tbl_Livro	Escreve	Tabela para cadastro dos autores dos livros
tbl_Editora	tbl_Livro	Publica	Cadastro de editoras



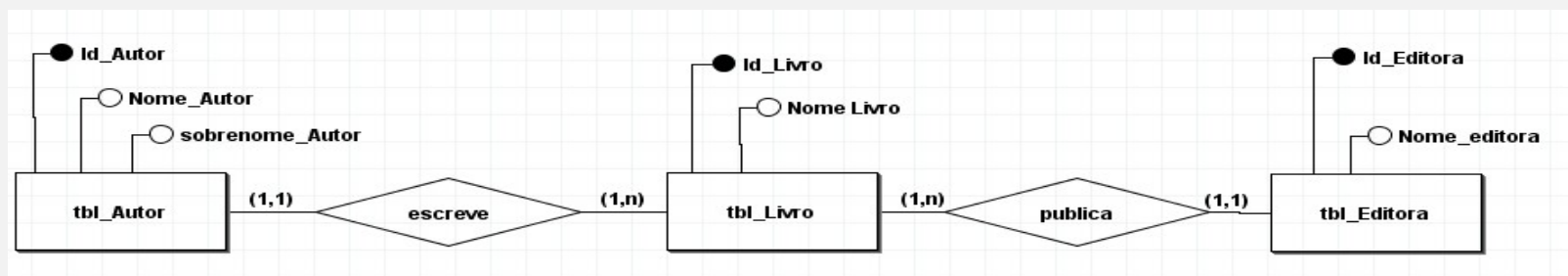
Exemplo de Dicionário de Dados

Tabela	Nome da Coluna	Tipo de Dados	Comprimento	Restrições	Valor Padrão	Descrição
tbl_Livro	ID_Livro	Inteiro	4 bytes	PK, NOT NULL	N/D	Número de identificação do livro, gerado automaticamente
	Nome_Livro	Caracteres	40 bytes	NOT NULL	N/D	N/D
	ID_Autor	Inteiro	4 bytes	FK	N/D	Nº de identificação do autor
	ID_Editora	Inteiro	4 bytes	FK	N/D	Nº de identificação da editora



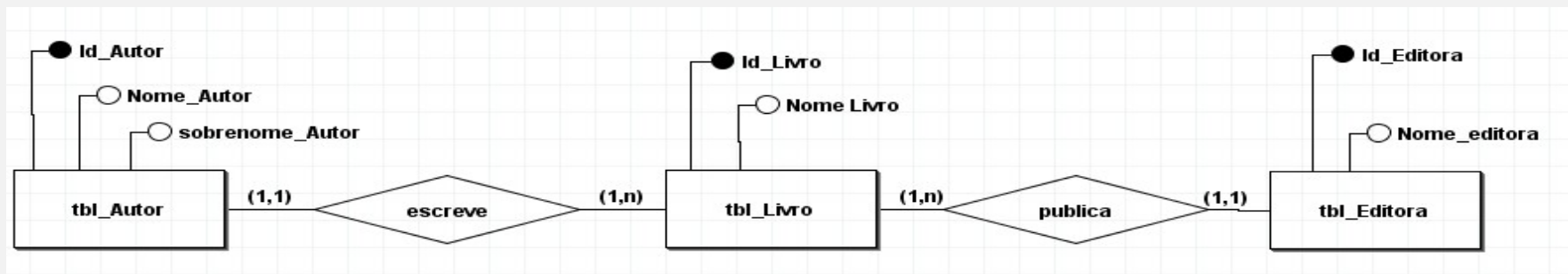
Exemplo de Dicionário de Dados

Tabela	Nome da Coluna	Tipo de Dados	Comprimento	Restrições	Valor Padrão	Descrição
tbl_Autor	ID_Autor	Inteiro	4 bytes	PK, NOT NULL	N/D	Número de identificação do autor, gerado automaticamente
	Nome_Autor	Caracteres	40 bytes	NOT NULL	N/D	Nome do autor
	Sobrenome_Autor	Caracteres	40 bytes	NOT NULL	N/D	Sobrenome do autor



Exemplo de Dicionário de Dados

Tabela	Nome da Coluna	Tipo de Dados	Comprimento	Restrições	Valor Padrão	Descrição
tbl_Editora	ID_Editora	Inteiro	4 bytes	PK, NOT NULL	N/D	Número de identificação da editora, gerado automaticamente
	Nome_Editora	Caracteres	40 bytes	NOT NULL	N/D	Nome da Editora



Exemplo de Dicionário de Dados

Relacionamento	Tabela 1 - FK	Tabela 2 - PK	Descrição
"Escreve"	tbl_Livro	tbl_Autor	Relacionamento que descreve qual autor escreve cada livro
"Publica"	tbl_Livro	tbl_Editora	Relacionamento que mostra qual editora publica cada livro

